

Analysis: Costume Change

Test set 1

Trying every possible costume on every dancer will be too slow, even for this test set. Therefore, we need another solution.

To solve this test set, we can first observe that this problem is equivalent to the following: Find the largest subset of dancers such that no two dancers in the subset have the same costume type and share the same row or column.

If we find such a subset, we can change the costume types of the dancers not in the subset so that they follow the rules. We can iterate through the dancers in any order (e.g. row-major). If the current dancer is in the subset, we leave the costume as it is. Otherwise, we pick a new costume type for that dancer. Since a dancer can be in the same row or column as at most $2N - 2$ other dancers, and since there are $2N$ costume types, it will always be possible to find a valid type to change to.

So, we can iterate through every subset of dancers and check whether there are two dancers in the subset having the same costume type and sharing the same row or column. This solution runs in $O(2^{N^2} \times N^2)$ time.

Test set 2

To solve this test set, we need a much more efficient way of finding the subset mentioned above. We can solve the problem independently for each costume type. Let $f(x)$ be the largest subset of dancers who are wearing a costume with type x , such that no two dancers in the subset share a row or column. Then the size of our desired subset will be $\sum f(i)$ for $-N \leq i \leq N$, $i \neq 0$.

How can we find $f(x)$? Let us create a bipartite graph as follows:

- $\{A_i\}$ is a set of vertices, where each vertex corresponds to a row.
- $\{B_j\}$ is another set of vertices, where each vertex corresponds to a column.
- If and only if the dancer in the i -th row and the j -th column wears a costume of type x , we add an edge connecting A_i and B_j .

To find the largest valid subset of dancers, we need to find the maximum independent set of this graph, since we can only pick one dancer for each row and column. We can use a [maximum cardinality bipartite matching algorithm](#) to solve this problem.

A simple maximum cardinality bipartite matching algorithm runs in $O(VE)$, where V is the number of vertices and E is the number of edges. Since $V = O(N)$ and the sum of E among all costume types is N^2 , this solution runs in $O(N^3)$, which is fast enough to solve test set 2.

Analysis: Falling Balls

Test set 1

We can narrow down the scope of Test set 1 with only a couple of insights:

- We cannot put any ramps in the leftmost and rightmost columns, so the balls dropped into those columns must fall straight down to the bottom. So, if either of those columns has no balls, the case is `IMPOSSIBLE`. (Later on in this analysis, it will become clear that this is the only way for a case to be impossible!)
- Suppose C has its maximum value of 5. Since we know the leftmost and rightmost columns must have at least one ball each, the only variation among cases comes from where the other 3 balls end up. It turns out that there are only 35 ways to partition 3 balls among 5 columns, and some of these cases are mirror reflections of each other, so there are really only 19 distinct cases to consider. When C is smaller than 5, there are even fewer cases.

So, we can experiment offline, generating potential ramp arrangements via simulation or by hand, until we are confident in our solutions for all cases, and then submit; since Test set 1 is Visible, we have little to lose. However, we can avoid this extra work, as follows...

Test set 2

Consider the paths that each ball might take through the toy, and notice that whenever two balls' paths intersect, those balls must end up in the same final column. (Because we can never have a `\` ramp immediately to the left of a `/` ramp, paths cannot cross over each other.)

So, for example, if your friend says there are exactly K balls in the leftmost column, they must have been the balls that were originally dropped into the K leftmost columns. Suppose that one of those balls had instead come from, e.g., the $(K+1)$ -th column; then the path taken by that ball would have crossed all of the paths taken by the balls in the K leftmost columns, so all $K+1$ of the balls would have ended up in the leftmost column, which is a contradiction.

With those observations in mind, let us think of the i -th column as both a *source* of one ball and a potential *target* of B_i balls (if $B_i > 0$). We can scan our friend's list from left to right, and when we encounter a positive B_i value, we allocate the B_i leftmost source columns that we have not used so far to target column i . So, for example, if we have the data `3 2 0 0 0 0 2 1`, we map the source column range $[1, 3]$ to target column 1, the source column range $[4, 5]$ to target column 2, the source column range $[6, 7]$ to target column 7, and the source column range $[8, 8]$ to target column 8.

Once we have this mapping, we can move on to designing the toy to ensure that every source column's ball will end up at the bottom of the appropriate target column. We will start with an empty top row, and add ramps and more rows as needed.

For each source column range, we check whether the left endpoint is to the left of the target. If it is not, we do nothing. Suppose, on the other hand, that it is L columns to the left of the target. Then we add $L \setminus$ ramps, starting in the top row and the left endpoint's column, and proceeding diagonally, moving one cell to the right and one cell down each time. Notice that this line of ramps will catch all balls in that source column range that need to move to the right, so we do not need to explicitly worry about those balls. Then, we draw similar diagonal lines of `/` ramps

running down and to the left from our right endpoints. Finally, if our bottom row is not empty, we add an empty bottom row, as the rules require.

Can we convince ourselves that this construction is optimal? We have already shown that the mapping of source columns to target columns is forced. Consider the largest absolute difference, in columns, between a target column and one of the endpoints of the range of source columns mapped to it. (In our example above, this value is $|5 - 2| = 3$.) The toy must have at least this many rows (in addition to the empty bottom row), because a ball can only interact with one ramp per row, and each ramp only moves a ball one cell in the desired direction. Observe that our construction uses exactly this many rows, plus the required empty bottom row. For our example, our method produces:

```
.. / . / \ ..  
./ ./ .....  
.. / .....  
.....
```

Notice that there is no danger of creating a `\ /` pattern within a range, since no target position could cause that.

Analysis: Graceful Chainsaw Jugglers

We can identify each juggler by a pair of integers (b, r) where $0 \leq b \leq \mathbf{B}$ and $0 \leq r \leq \mathbf{R}$. The pair $(0, 0)$ does not identify a valid juggler, although we could assume that it does and, since we can always add $(0, 0)$ to any valid configuration that does not contain it, just solve the modified problem and subtract 1 to obtain the final answer. We can then focus on finding the size of the largest subset of $V = [0; \mathbf{B}] \times [0; \mathbf{R}]$ such that the sum of the first component of each element is $\mathbf{B}$, and the sum of the second component of each element is $\mathbf{R}$. Let us call b, r -valid to a set of distinct pairs that fulfills the sum conditions for specific b and r .

Test set 1

For Test set 1, we can first notice that the size of V is small. Therefore, we can do a dynamic programming iterating through V and deciding, for each pair, whether to include it or not. As part of the state we have to also include how many blue and red jugglers we can still include.

Therefore, we enumerate V in any order $v_1, v_2, \dots, v_{(\mathbf{B}+1) \times \mathbf{R}+1}$, and then we compute a function $f(i, b, r)$ defined as the size of the largest b, r -valid subset of $\{v_1, v_2, \dots, v_i\}$.

We can see that f has also a recursive definition as follows:

- $f(0, 0, 0) = 0$.
- $f(i + 1, b, r) = \max(f(i, b, r), 1 + f(i, b - \text{left}(v_i), r - \text{right}(v_i)))$ for all b, r, i .

The definition above is incomplete because it's not checking for indefinities. A simple solution is to just define every undefined case as -infinity, or for this particular problem, $-\mathbf{B} - \mathbf{R}$.

Memoizing this recursive definition leads to an implementation that takes time proportional to the size of the domain of f , which is $(\mathbf{B}+1)^2 \times (\mathbf{R}+1)^2$, which is likely fast enough to solve Test set 1. If your implementation and language of choice are both on the slow side, it's possible that you need some additional insight to make it through the time limit. One simple trick is that the function f actually solves all possible test cases, since the set V is fixed. So, you can reuse the memoization table for all test cases and save 99% of the work compared to solving each test case from scratch. You can even compute the entire table before submitting, and include it in your source code, although you should be wary of this strategy in general since it might cause your source file to exceed the 1 MB limit specified in our rules.

Test set 2

For Test set 2, we will definitely need the additional insight of not resetting the memoization table for each test case. However, the important additional insight is more specific to the problem.

First, let us define weak- b, r -valid sets as sets of distinct pairs of non-negative integers such that the sum of the values of the left sides of each pair is less than or equal to b , the sum of the values of the right sides of each pair is less than or equal to r . Of course, every b, r -valid set is also a weak- b, r -valid set, but the converse is not true.

One important property of weak- b, r -valid sets is that for fixed b and r , the size X of the largest weak- b, r -valid set is equal to the size Y of the largest b, r -valid set. We can prove that by noticing: (1) $Y \leq X$ because every b, r -valid set is a weak- b, r -valid set; and (2) $X \leq Y$ because if we let S be a weak- b, r -valid set of size X and then take an element (i, j) of S such that i is maximum, and j is maximum among those with maximum i , we can construct $S' = S - \{(i, j)\} \cup \{(i$

+ db, j + dr)) where db and dr are the difference between the sums of the left/right values of the pairs and b/r, respectively. By construction, S' is b,r-valid, and by the maximality of the choice of (i,j), (i + db, j + dr) is not in S - {(i,j)}, and thus, the sizes of S and S' are equal.

Let us now define minimal-weak-b,r-valid sets as weak-b,r-valid sets S such that for any (i,j) in S with i > 0, (i-1, j) is also in S, and analogously, for any (i,j) in S with j > 0, (i, j-1) is also in S. Similarly to the above, we can prove that the size of the largest minimal-weak-b,r-valid set for fixed b and r is the same as the size of the largest weak-b,r-valid sets by constructing a minimal-weak-b,r-valid sets of size X from a given weak-b,r-valid set S of size X. Let (i,j) in S be such that the minimality of the definition is violated (if such exists). Then, replace (i,j) in S for either (i-1,j) or (j-1,i) as appropriate to obtain another weak-b,r-valid set of the same size. This step strictly decreases the sum of the values of all pairs in S, thus, at some point, there are no such (i,j) to choose for and we successfully obtained a minimal-weak-b,r-valid set of size X.

The problem that remains is to find the size of the largest minimal-weak-b,r-valid set for given b and r. By the minimality condition, if (i,j) is in such largest set, so are (i-1, j), (i-2,j), (i,j-1), (i,j-2), (i-1, j-1), To formalize, (i,j) being present in a minimal-weak-b,r-valid set prescribes the presence of (i+1) × (j+1) pairs including itself. We can bound the sum of the left side values of all those pairs by (j+1) × (1 + 2 + ... + i) = (j+1) × i × (i+1) / 2, and analogously the sum of the right sides by (i+1) × j × (j+1) / 2. This means that instead of the set V from the solution above, we can use V', composed of the pairs in V that are under these bounds. This makes the size of V' be approximately $O(\mathbf{B}^{1/3} \times \mathbf{R}^{1/3})$ when **B** and **R** are close, which is a big reduction from the size of V which is $O(\mathbf{B} \times \mathbf{R})$. The overall complexity of the resulting algorithm for all test cases is then $O(\mathbf{B}^{4/3} \times \mathbf{R}^{4/3})$, which is fast enough to pass Test set 2.

Analysis: Gridception

Test set 1

This test set can be solved using complete search. We can try enumerating every possible connected pattern. For each possible connected pattern, we check whether that pattern exists in the grid which has been deepened twice. Among all connected patterns which exist in the grid which has been deepened twice, we take the one with the largest number of cells. The correctness of this algorithm will be proved later in this section.

Note that it is not sufficient to check the pattern in the grid which has been deepened only once. The first sample case in the problem description shows that there is a pattern which does not exist in the grid which has been deepened once, but the pattern exists in the grid which has been deepened more than once.

Why is it sufficient to check the pattern in the grid which has been deepened twice? We can first observe that a grid which has been deepened X times consists of blocks of cells. Each block is a square of cells of the same color with side length 2^X . This does mean any pattern of size at most 3×4 can fit in the same block in the grid which has been deepened twice, whereas this might not be the case in the grid which has been deepened once.

Moreover, any pattern of size at most 3×4 overlaps with at least one and at most four blocks in the grid which has been deepened twice. This is also the case in the grid which has been deepened more than twice. Therefore, the set of patterns of size at most 3×4 which exists in the grid which has been deepened twice is equivalent to the set of patterns of size at most 3×4 which exists in the grid which has been deepened X times for any $X > 2$. Therefore, it is sufficient to check the pattern in the grid which has been deepened twice.

There are at most $O(2^{R \times C})$ patterns. Therefore, there are not more than $O(2^{R \times C})$ possible connected patterns. For each pattern, we can first check whether it is connected, and if it is, we can then check whether that pattern exists in the grid which has been deepened twice in $O(R \times C)$ time. Therefore, the total complexity of this solution is $O(2^{R \times C} \times R \times C)$ which is fast enough to solve test set 1.

Test set 2

Since R and C can be up to 20, the exponential solution will not run in time. Therefore, we must not enumerate every possible connected pattern. To solve this test set, we can first observe that a block of cells in the grid which has been deepened at least a googol times will have a side length larger than the size of any possible pattern.

The observation in the previous paragraph ensures that every possible pattern can overlap with at most four blocks of cells in the deepened grid. This means that Codd's pattern needs to be divisible into four quadrants by a horizontal and a vertical line where each cell of the pattern in the same quadrant has the same color. Moreover, that particular combination of four colors has to exist in the original grid.

Therefore, we can consider every possible quadrant center and combination of colors (there are up to $O(2^4 \times R \times C)$). For each quadrant center and combination of colors, we want to get the largest connected component where each cell in this connected component has the same color as the color assigned to the quadrant it belongs to.

For each possible quadrant center and combination of colors, we need $O(\mathbf{R} \times \mathbf{C})$ time to find the largest connected component. Therefore, this solution will run in $O(2^4 \times \mathbf{R}^2 \times \mathbf{C}^2)$ time.